FACTORY OPERATIONS V3

Meta App Factory — Resilience Anchor V3.0

Compiled: March 13, 2026 | Authorized Credential: Antigravity_Full_v2

19	86	193	472
AGENTS	N8N WORKFLOWS	RETROFITTED FILES	INDEX LINES

01 Resilience Architecture

V3 Core DNA

Every file in the factory inherits the V3 resilience DNA: `safe_post()` routing, `auto_heal()` recovery, and `diagnose()` self-repair.

📁 → 🔍 → 🛡️ → ☁️ → 📋

Auto-Heal Layer

MODULE	FUNCTION	LOCATION

`auto_heal.py`	healed_post(), auto_heal(), diagnose()	Factory root
`circuit_breaker.py`	CircuitBreaker pattern (5 failures → open)	Factory root
`local_state_manager.py`	Offline buffer → sync on reconnect	Factory root
`resilience_config.json`	Cloud health, retry limits, watchdog URL	Factory root
`api.py _call_n8n_agent`	@auto_heal(project="WarRoom") — War Room agent calls	Factory root

Self-Healing Nervous System

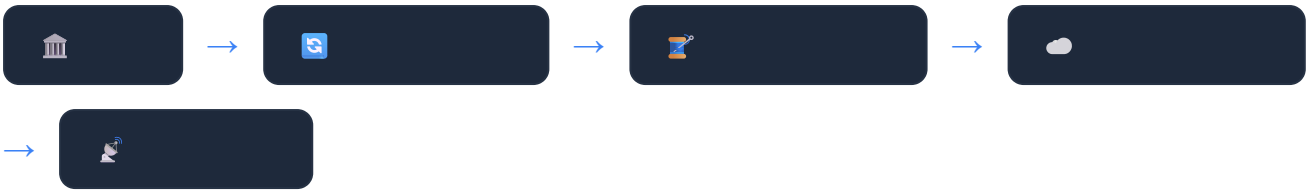
`nerve_center.py` — Closed-loop autonomic recovery engine. SCAN → ANALYZE → ACT → LOG every 300s.

REMEDY	ACTION
AUTH_EXPIRED	Credential refresh from vault/env
MALFORMED_JSON	Sanitize and retry
GATEWAY_TIMEOUT	Exponential backoff + retry
RATE_LIMITED	Exponential backoff + retry
CONNECTION_REFUSED	Wait + retry (3x)
CIRCUIT_OPEN	Reset breaker after verification

02 War Room — Adversarial Boardroom

Live n8n Agent Routing

The War Room is wired directly to the n8n Cloud specialist agent webhooks. Every agent message is a **real AI response** — not a template. The `_live_debate()` function calls all 4 agents in parallel via `ThreadPoolExecutor`, then streams their responses into the WebSocket feed.



Agent Webhook Registry

AGENT	WEBHOOK	PROMPT FOCUS	RESILIENCE
👔 CEO	`elite-council`	Strategic direction, competitive advantage	@auto_heal
📢 CMO	`cmo-v2`	Market analysis, audience, GTM timing	@auto_heal
💰 CFO	`cfo-v2`	Unit economics, burn rate, ROI	@auto_heal
🔍 CRITIC	`critic-v2`	Weakness identification, score 1-10	@auto_heal

V3 Resilience Compliance

Every War Room agent call is wrapped with `@auto_heal(project="WarRoom", max_retries=3)` :

LAYER	BEHAVIOR
Retry	Exponential backoff: 2s → 4s → 8s
Diagnosis	On 3x failure: watchdog + credential + buffer check
Logging	All heal events → `auto_heal_log.json`
Fallback	⚠️ [Cached] generic response shown in UI

Score Parsing	Critic's X/10 score → Persuasion Meter auto-update

WebSocket Events

AGENT	ROLE	COLOR
👔 CEO	Strategic direction + executive decisions	Blue
📢 CMO	Market positioning + audience research	Purple
💰 CFO	Financial model + ROI projections	Green
🔍 CRITIC	Quality assurance + devil's advocate	Red
📐 ARCHITECT	System design + implementation	Cyan

Socratic Challenger Protocol (Phase 3)

The dialectical challenge engine enforces quality gates on all proposals.

📋 → 🔍 → ⚠️ →

🏛️ → ✅ 🚨

Convince Logic

SIGNAL TYPE	EXAMPLES	IMPACT
Strong Evidence	data shows, validated, benchmark, A/B test, ROI	+1.5 per signal (max +5)
Weak Reasoning	I think, probably, gut feeling, trust me	-0.5 per signal
Coverage	Addressing each weakness category	+2 max bonus

Substance	100+ words = +3, 50+ = +2, 20+ = +1	Length bonus

Override Risk Levels

LEVEL	GAP	CONSEQUENCE
LOW	≤ 1 pt	Minor gaps — proceed with monitoring
MEDIUM	≤ 3 pts	Substantive objections unaddressed
HIGH	≤ 5 pts	Critical weaknesses bypassed — 30-day audit required
CRITICAL	> 5 pts	Structural risks overridden — full audit within 30 days

03 Venture Studio Engine

Global Skills (.system_core)

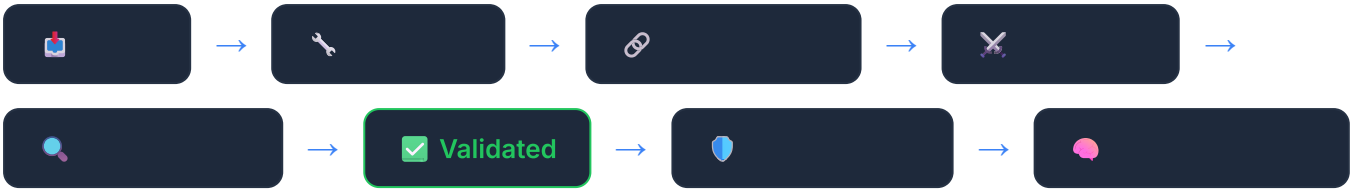
The `.system_core` package is the canonical inheritance layer. Every child project symlinks into it, gaining immediate access to factory-grade capabilities without code duplication.

MODULE	CAPABILITY	STATUS
`pii_masker.py`	PII & secrets redaction (paths, API keys, tokens, IPs)	ACTIVE
`visual_engine.py`	PDF, DOCX, PPTX, XLSX generation with brand enforcement (Nano Banana 2)	ACTIVE
`market_crawler.py`	Tavily search, competitive scanning, market_intel.db	ACTIVE
`deploy_shield.py`	Deployment sanitization — strips keys, logs, PII before third-party delivery	ACTIVE
`wisdom_accumulator.py`	Institutional memory — captures winning War Room logic into global_wisdom.db	ACTIVE

`kill_switch.py`	Emergency API key protection — panic/restore on spending breach	ACTIVE

Venture Lifecycle

Every venture follows this end-to-end lifecycle — from raw ingest through the adversarial boardroom to wisdom archival.



Lifecycle Stages

STAGE	MODULE	OUTPUT
1. Ingest	`incoming_watcher.py`	New files detected in `projects/*/incoming/`
2. Bootstrap	`bootstrap_project.py`	Project folder with Soul, brand, and `.system_core` links
3. Symlink Core	`.system_core/`	Child inherits all 6 global skills via filesystem links
4. War Room	`api.py → _live_debate()`	CEO / CMO / CFO / Critic responses via n8n webhooks
5. Socratic Gate	`socratic_challenger.py`	3 weaknesses + persuasion loop + override logging
6. Validated	`socratic_challenger.py`	Status: User-Validated or Commander-Override
7. Deploy Shield	`deploy_shield.py`	Sanitized `_deploy/` bundle + `deploy_manifest.json`
8. Wisdom Archive	`wisdom_accumulator.py`	Winning logic appended to `global_wisdom.db`

The Wisdom Vault

The Wisdom Vault is the factory's institutional memory. Every time a Commander successfully convinces the Critic in the War Room — achieving a CONVINCED verdict — the winning logic is automatically captured and converted into "Global Core" intelligence.

How Overrides Become Intelligence

The `wisdom_accumulator.py` module hooks directly into the Socratic Challenger's `analyze_response()` method via monkey-patching. When a verdict of **CONVINCED** is returned:

STEP	WHAT HAPPENS	WHERE IT LIVES
1. Intercept	The hooked `analyze_response()` detects `verdict == "CONVINCED"`	`wisdom_accumulator.py`
2. Extract	Challenge ID, proposal text, the Commander's reasoning, and all analysis metrics are packaged	In-memory
3. Score	The combined score (original Critic + reasoning quality), strong/weak signal counts, and weakness coverage ratio are recorded	In-memory
4. Persist	A JSON-line entry is appended to `global_wisdom.db` — one line per win, append-only, no data loss	`.system_core/global_wisdom.db`
5. Recall	Future debates can query the vault: `--query "scalability"` returns all prior victories addressing that category	CLI / API
6. Aggregate	`--stats` reveals patterns: avg scores, strongest signal types, full-coverage win rates	CLI / API

This means the factory **learns from its own debates**. Over time, the Wisdom Vault reveals which argument structures work, which evidence types are most

persuasive, and which weakness categories the Commander has mastered.

The Sanitization Protocol

Before any application folder is shared with a third party, investor, or external team, it must pass through the **Deploy Shield** — a multi-layered sanitization engine that ensures zero internal data leakage.

Three-Layer Sanitization

LAYER	WHAT IT DOES	WHY IT MATTERS
Layer 1: Structural Pruning	Removes entire directories (`__pycache__` , `.git` , `node_modules` , `socratic_logs` , `.audit_snapshots`) and internal log files	Eliminates debug artifacts, version history, and internal audit trails that contain operational fingerprints
Layer 2: Key Stripping	Rewrites every `*_KEY` , `*_TOKEN` , `*_SECRET` value in `.env` files to `REDACTED`	API keys and tokens are replaced — not deleted — so the file structure is preserved for the recipient to fill in their own credentials
Layer 3: PII Masking	Runs `PIIMasker` on all text-based source files (`.py` , `.js` , `.json` , `.md` , `.html` , `.yaml`)	Catches stray file paths, IP addresses, email addresses, and hardcoded tokens embedded in source code

The output is a clean `_deploy/` staging folder with a `deploy_manifest.json` that logs every file shipped, every redaction applied, and the total sanitization statistics. This manifest serves as a verifiable audit trail.

Budget Kill-Switch

The Kill-Switch is the factory's financial circuit breaker. It monitors the n8n execution rate and automatically disables all API keys if hourly spending exceeds a configurable threshold — preventing runaway costs from automated workflows.

Automated Protection Chain

📊 → → status == "critical" (≥ 90%)

→ → 🔒

EVENT	ACTION	RECOVERY
Rate ≥ limit/hour	Backup `.env` → `.env.panic_backup`, rewrite all key values to `DISABLED_BY_KILLSWITCH`	`python kill_switch.py --restore`
Manual panic	`python kill_switch.py --panic` — same as above but triggered by operator	Same restore command
Status check	`python kill_switch.py --status` — shows ARMED (🟢) or TRIPPED (🔴)	N/A (read-only)

The kill-switch is **non-destructive**: the original `.env` is always backed up first, and restore is a single command. All panic/restore events are logged to `kill_switch_log.json` (last 100 events retained).

Single App Maintenance

Child apps inherit from `.system_core` via symbolic links created by `bootstrap_project.py`. When updating a child app, follow these rules to avoid breaking the inheritance chain.

Safe Update Protocol

RULE	DESCRIPTION
DO Edit child-local files	Any file inside the child's own directory (e.g. `MyApp/server.py`) is safe to edit, replace, or delete
DO Update `.system_core` centrally	Edits to `.system_core/*.py` at the factory root automatically propagate to all child apps via symlinks — one fix benefits every project

DO Use `bootstrap_project.py`	Always re-run bootstrap if adding new core modules — it will create any missing symlinks without disrupting existing ones
DON'T Delete symlink targets	Never delete a file in `.system_core/` if any child still references it — check with `dir /s *.symlink` first
DON'T Replace symlinks with copies	Replacing a symlink with a local copy breaks the inheritance chain — that child will no longer receive upstream fixes
DON'T Move the project folder	Symlinks use relative paths — moving the child folder to a different depth level breaks the `..` resolution

Child App Directory Structure

PATH	TYPE	SOURCE
`MyApp/`	Directory	Child project root
`MyApp/soul/`	Directory	Project Soul — mission, context, brand identity
`MyApp/system_core → ../.system_core`	Symlink	Factory global skills (PIIMasker, VisualEngine, etc.)
`MyApp/server.py`	File	Child's own application code
`MyApp/.env`	File	Child's own environment variables
`MyApp/incoming/`	Directory	Ingest folder — watched by `incoming_watcher.py`

Project Bootstrapper

`bootstrap_project.py` creates new ventures with symbolic links to `.system_core`, project Soul, and brand identity. Run `python bootstrap_project.py <ProjectName>` to scaffold a new child app.

Brand Guardian

`brand_guardian.py` audits files for brand compliance (fonts, colors, mission alignment) with 0-100 scoring. Runs automatically during the Socratic challenge phase.

04 Phase 4 — Boardroom Integration

Incoming File Watcher

`incoming_watcher.py` monitors `projects/*/incoming/` and auto-seeds War Room boardroom sessions + Socratic challenges.

ENDPOINT	FUNCTION
`POST /api/incoming/scan`	One-shot scan of incoming directories
`POST /api/incoming/watch`	Start background watcher (30s poll)
`POST /api/n8n/archive`	Archive V1/TEST/OLD workflows
`POST /api/audit/master_index`	Full Socratic Audit on MASTER_INDEX.md

n8n Optimization

Antigravity_Full_v2 key manages 86 workflows. Legacy archiver identified 2 workflows matching archive criteria. Critic V2 confirmed as sole arbiter across 14 quality workflows.

MASTER_INDEX Audit Results

METRIC	VALUE
Total Lines	472
Sections	30+
Self-Healing Cycles	14

Deployed Modules	10+
Error Entries	5
Fresh Boots	4

05 API Surface

Core Endpoints

ENDPOINT	METHOD	PHASE
`/api/health`	GET	Core
`/api/registry`	GET	Core
`/api/chat`	POST	Core
`/ws/warroom`	WS	Phase 2
`/api/warroom/seed`	POST	Phase 2
`/api/warroom/challenge`	POST	Phase 3
`/api/warroom/convince`	POST	Phase 3
`/api/warroom/force_proceed`	POST	Phase 3
`/api/incoming/scan`	POST	Phase 4
`/api/incoming/watch`	POST	Phase 4
`/api/n8n/archive`	POST	Phase 4
`/api/audit/master_index`	POST	Phase 4

06 Security & Credentials

KEY	SCOPE	STATUS
Antigravity_Full_v2	n8n Cloud API (all workflows)	ACTIVE
Fernet AES-128	.env encryption at rest	DEPLOYED
PBKDF2 (600K iterations)	Key derivation	ACTIVE
.env Read-Only	Child process protection	ENFORCED

07 Deploy Shield — Sanitization SOP

Purpose

Automates the sanitization of any App folder before third-party delivery. Strips internal logs, API keys, PII, and debug artifacts to produce a clean release bundle with a verifiable manifest.

Sanitization Rules

RULE	ACTION	TARGETS
Directory Pruning	Remove entirely	`__pycache__`, `.git`, `node_modules`, `.venv`, `socratic_logs`, `.audit_snapshots`
Log File Removal	Skip during copy	`auto_heal_log.json`, `socratic_audit.json`, `n8n_execution_log.json`, `*.log`, `*.bak`
.env Key Stripping	Rewrite values to `REDACTED`	All `*_KEY`, `*_TOKEN`, `*_SECRET` variables

PII Masking	Run `PIIMasker` on text files	`.py`, `.js`, `.json`, `.md`, `.html`, `.yaml` + more

CLI Usage

COMMAND	EFFECT
`python .system_core/deploy_shield.py MyApp`	Sanitize MyApp → `_deploy/MyApp/`
`python .system_core/deploy_shield.py MyApp -o ./release`	Custom output directory

Output includes `deploy_manifest.json` with file count, redaction stats, and shipped file list.

08 Wisdom Accumulator — Institutional Memory SOP

Purpose

After every successful "Persuasion" in the War Room, the winning logic is automatically captured into `.system_core/global_wisdom.db` — a JSON-lines knowledge base that grows institutional memory over time.

🏛️ → 🔍 → 💬 →

✅ CONVINCED → 🧠 → 📚

Wisdom Entry Format

FIELD	TYPE	DESCRIPTION

`ts`	ISO timestamp	When the wisdom was captured
`challenge_id`	string	Socratic challenge ID (e.g. CHG-0001)
`proposal`	string	Original proposal text (first 500 chars)
`winning_reasoning`	string	The Commander's winning argument
`combined_score`	float	Final combined score (Critic + Reasoning)
`strong_signals`	int	Count of evidence-backed reasoning signals
`weaknesses_addressed`	string	Coverage ratio (e.g. "3/3")

Hook Integration

The accumulator patches the Socratic Challenger's `analyze_response()` method at startup:

STEP	ACTION
1. Init	`wisdom.hook_into_socratic(get_challenger())`
2. On CONVINCED	Auto-fire `record_wisdom()` with challenge context
3. Storage	Append JSON line to `.system_core/global_wisdom.db`
4. Query	`--query "keyword"` searches across all captured reasoning
5. Stats	`--stats` shows win rates, avg scores, signal patterns

09 Kill-Switch — Emergency Spend Protection SOP

Purpose

A "Panic" function that temporarily disables all Antigravity API keys if the hourly execution rate exceeds a configurable threshold. Linked to `n8n_budget_guard.py` for automated cost protection.

📊 → ⚠️ → 🚨 check_and_panic() →
💾 → 🔒 → 📋

Kill-Switch States

STATE	INDICATOR	DESCRIPTION
ARMED	🟢	Monitoring active, ready to trip if threshold exceeded
TRIPPED	🔴	API keys disabled, .env backed up to .env.panic_backup
RESTORED	🔄	Keys restored from backup, system operational

Integration with Budget Guard

`n8n_budget_guard.py` → `check_budget()` → on `"critical"` status → imports `kill_switch.get_kill_switch()` → fires `check_and_panic(hourly_limit)`.

CONFIG	SOURCE	DEFAULT
Hourly limit	`resilience_config.json` → `kill_switch.hourly_limit`	500 exec/hour
Env files	Auto-detected: Factory root + Alpha_V2_Genesis	—
Log	`kill_switch_log.json` (last 100 events)	Factory root

CLI Usage

COMMAND	EFFECT
`python .system_core/kill_switch.py --status`	Check current state (ARMED / TRIPPED)
`python .system_core/kill_switch.py --limit 500`	Check rate, auto-panic if exceeded
`python .system_core/kill_switch.py --panic`	Force-trip (disable all keys now)
`python .system_core/kill_switch.py --restore`	Restore keys from backup